SAMSUNG SW AI ACADEMY FOR YOUTH

對外秘

프로젝트 명세서

멀티모달 인식 통합과 VLM 기반

의도 파악 AI Workflow 개발

(AI 코어 프로젝트)

“멀티모달 인식 모델 연동”

2 회차

對外秘

목차

1. 멀티모달 인식 모델 연동 3

1.1 목표 3

1.2 준비사항 3

1.3 작업 순서 4

1.4 요구사항 4

1.5 참고자료 7

1.6 프로젝트 결과 8

對外秘

1. 멀티모달 인식 모델 연동

1.1 목표

이번 '멀티모달 인식 모델 연동' 프로젝트의 목표는 다음과 같습니다.

- YOLO·VLM 인식 결과를 LLM 이 읽을 Text 규격으로 고정한다.
- MediaPipe 로 손 제스처를 인식하고 가리키는 방향을 Text 로 변환한다.
- 제공된 웹 대시보드 화면의 요청을 처리하는 서버 API 를 구현한다.
- LLM 이 세 인식 결과 Text 와 사용자 질문으로 상황을 설명하도록 통합한다.

1.2 준비사항

1) 프로젝트 구조

- 이번 프로젝트는 1 회차(PoC 제작 및 Object Detection 파인튜닝)의 PoC 서버(이미지 in → 텍스트 out)를 이어받아 손 제스처를 세 번째 입력으로 더하고, 웹 대시보드에서 LLM 답변까지 잇는 구조로 진행한다.

  먼저 YOLO 결과와 VLM 설명을 정해진 Text 규격으로 바꾸고, MediaPipe 로 손 제스처의 방향을 판별한다. 이후 완성본으로 제공되는 대시보드 화면(index.html)의 요청을 처리하는 서버를 만들고, LLM(Qwen3-4B)이 세 출처의 Text 를 종합해 답하도록 연결한다.

  - mediapipe 패키지·제스처 모델·LLM 가중치 추가 설치 및 환경 체크
  - YOLO 결과 '이름 N 개' 요약 및 VLM 출력 규격화 (긴 변 448px 입력)
  - MediaPipe 손 제스처 인식 및 가리키는 방향 판별
  - 웹 대시보드 서버 구현 (YOLO·VLM API)
  - LLM 통합 답변, JSON 방어 파싱, 대시보드 연결

2) 사용 데이터

- 이번 프로젝트에서는 별도의 학습 데이터셋을 직접 구축하지 않는다. 제공되는 샘플 이미지(sample.jpg, pointing_up.jpg, victory.jpg)와 웹캠 스냅샷을 입력으로 사용하여 인식 결과와 LLM 답변을 확인한다.

3) 개발언어 및 툴

- Python 3.12 : 인식 결과 변환, 대시보드 서버, LLM 통합 코드를 구현하는 데 사용

- HTML / JavaScript : 완성본으로 제공되는 대시보드 화면(index.html)과 브라우저 제스처 인식
- Chrome / Edge : 대시보드 접속 및 웹캠 입력

4) 필수 라이브러리 / 오픈소스

- Ultralytics (YOLO11n) : 객체 인식
- llama-cpp-python (0.3.34) : VLM(Qwen2.5-VL-3B-Instruct)과 LLM(Qwen3-4B) GGUF 추론
- MediaPipe (GestureRecognizer) : 손 랜드마크 21 점과 제스처 인식
- FastAPI / Uvicorn : 대시보드 화면과 인식·LLM API 를 제공하는 서버

1.3 작업 순서

1) mediapipe 와 제스처 모델·LLM 가중치를 추가 설치하고 환경을 확인한다

2) YOLO 결과와 VLM 설명을 LLM 입력용 Text 규격으로 변환한다

3) MediaPipe 로 손 제스처를 인식하고 가리키는 방향을 판별한다

4) 웹 대시보드 서버를 구현해 웹캠 스냅샷의 YOLO·VLM 결과를 화면에 표시한다

5) LLM 에 출처별 인식 Text 와 질문을 넣어 통합 답변을 만든다

6) JSON 방어 파서를 구현하고 대시보드에 LLM 답변을 연결한다

1.4 요구사항

본 프로젝트는 서로 다른 인식 모델의 출력을 하나의 Text 규격으로 통합하고, LLM 이 이를 종합해 상황을 이해하는 과정을 단계적으로 학습하는 데 목적이 있다.

■ **요구사항 예시 (참고용)**

- 아래의 내용을 참고하여 추가적인 아이디어에 대해 요구사항을 추가 또는 수정하여 기능을 구현합니다. 단, 필수 기능은 반드시 구현해야 하며 수정할 수 없습니다.

기능적 요구사항

번호	분류	요구사항명	요구사항 상세	우선순위
F201	기능	인식 결과 Text 규격화	YOLO 결과 '이름 N개' 요약과 VLM 한 문장·장애물 유무 규격	필수
F202	기능	MediaPipe 손 제스처 방향 인식	제스처 인식과 손목 → 검지 끝 벡터로 방향 판별	필수
F203	기능	웹 대시보드 서버 구현	제공 화면의 요청을 처리하는 YOLO·VLM API 서버	필수
F204	기능	LLM 상황 통합 답변	출처별 인식 Text 와 질문으로 LLM 답변 생성, JSON 방어 파싱, 대시보드 연결	필수
F205	기능	손 제스처 인식 확장 (손가락 개수)	회전에 강한 손가락 개수 인식과 실사용 웹캠 환경 검증으로 제스처 Text 확장	심화
F206	기능	출처 충돌을 감지하고 알리는 LLM 답변	YOLO·VLM 결과 불일치를 코드로 감지하고 VLM 원문을 인용해 답하는 기능	심화

1. 기본 기능 (필수)

1) 인식 결과 Text 규격화

① 요구사항 번호 : F201

② YOLO 검출 결과에서 좌표·conf 를 버리고 'person 4 개, bus 1 개' 처럼 많은 순 '이름 N 개' 형식의 한 문장으로 요약한다.

③ VLM 입력을 긴 변 448px 로 줄이고, 출력은 '한국어 한 문장 + 장애물 있음/없음' 규격으로 프롬프트·max_tokens·코드(normalize·check) 세 곳에서 맞춘다.

2) MediaPipe 손 제스처 방향 인식

① 요구사항 번호 : F202

② GestureRecognizer 로 제스처 카테고리·왼/오른손·랜드마크 21 점을 인식한다.

③ 손목(0) → 검지 끝(8) 벡터로 가리키는 방향을 판별하고, 거울 화면이면 좌우만 뒤집는다.

④ 결과를 '가리키는 방향: 위' 형식의 Text 로 바꾸고, 판단할 수 없으면 '손

없음'·'인식 불가' 로 표시한다.

3) 웹 대시보드 서버 구현

① 요구사항 번호 : F203

② 제공된 화면(index.html)과 제스처 파일을 서빙하고, 웹캠 스냅샷을 받아 YOLO 박스와 VLM 장면 설명을 돌려주는 API(/api/yolo-detect, /api/vlm-describe)를 구현한다.

③ 추론은 run_in_threadpool 로 보내 YOLO·VLM 요청이 동시에 처리되게 하고, 오류는 {"error": "코드"} 로 응답해 한 모델이 실패해도 나머지 결과는 표시되게 한다.

4) LLM 상황 통합 답변

① 요구사항 번호 : F204

② Qwen3-4B 에 [YOLO 객체 인식]·[VLM 장면 설명]·[손 제스처] 출처별 블록과 [사용자 질문]을 넣고, 근거 충돌 규칙(개수는 YOLO, 상황은 VLM, 방향은 손)을 시스템 프롬프트에 정한다.

③ <think> 블록·앞뒤 잡글·잘린 출력에도 항상 같은 모양의 결과를 돌려주는 JSON 방어 파서를 구현한다.

④ /api/llm-answer 를 추가해 대시보드 답변창에 LLM 답변이 표시되게 한다.

2. 심화 기능 (선택)

필수 기능을 모두 구현한 후에는 프로젝트를 한 단계 확장하는 아래 심화 기능에 도전해보자.

1) 손 제스처 인식 확장 (손가락 개수)

① 요구사항 번호 : F205

② 손끝이 중간 관절보다 손목에서 더 먼지를 기준으로 펼친 손가락 개수(0~5)를 세고(엄지는 새끼 뿌리 기준), 손을 90°·180° 돌려도 같은 값이 나오게 한다.

③ 제스처 Text 를 'Victory, 가리키는 방향: 위, 손가락 2 개' 형식으로 확장하고,

LLM 입력 블록과 시스템 프롬프트에 반영한다.

④ 직접 찍은 손 사진(3 장 이상)으로 방향·손가락 개수를 검증하고, 좌우 반전(--mirror) 적용 여부를 확인해 결과를 정리한다.

2) 출처 충돌을 감지하고 알리는 LLM 답변

① 요구사항 번호 : F206

② YOLO 요약과 VLM 설명에서 사람 수 같은 개수 정보를 추출해 출처끼리 다른지 코드로 감지하고, 결과를 [출처 비교] 블록으로 LLM 입력에 추가한다.

③ 충돌이 있으면 개수는 YOLO 를 따르되 VLM 원문을 그대로 인용해 차이를 함께 알리는 규칙을 시스템 프롬프트에 추가한다 ('다르면' 같은 판단은 LLM 에 맡기지 않는다).

④ 충돌 / 비충돌 입력 여러 개로 답변을 비교하고, 새 프로세스로 다시 실행해도 같은 답이 나오는지 확인한다.

1.5 참고자료

- MediaPipe Gesture Recognizer 공식 문서
  https://ai.google.dev/edge/mediapipe/solutions/vision/gesture_recognizer
- MediaPipe Hand Landmarker 공식 문서
  https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker
- Qwen3-4B GGUF 모델
  https://huggingface.co/Qwen/Qwen3-4B-GGUF
- llama-cpp-python 공식 저장소
  https://github.com/abetlen/llama-cpp-python
- FastAPI 공식 문서
  https://fastapi.tiangolo.com/

API 명세				
번호	메서드	경로	요청 → 응답(200) / 오류	비고
A1	GET	/	대시보드 화면(index.html)을 돌려준다	2회차에서 제작 완료
A2	GET	/mediapipe/...	브라우저 제스처 인식 파일(vision_bundle.mjs, wasm, gesture_recognizer.task) / 404	2회차에서 제작 완료
A3	POST	/api/yolo-detect	JPEG 바이트 → {"objects": [{label, confidence, box}]} / 400 empty_image·invalid_image, 500 yolo_failed	1회차에서 제작 완료, 2회차에서 응답 형식 변경
A4	POST	/api/vlm-describe	JPEG 바이트 → {"description": 한 문장 + 장애물 있음/없음} / 400 empty_image·invalid_image, 500 vlm_failed	1회차에서 제작 완료, 2회차에서 오류 형식 변경
A5	POST	/api/llm-answer	{prompt, yolo_summary, vlm_description, gesture_direction} → {need_more_info: false, message, user_content} / 400 invalid_json·empty_prompt, 500 llm_failed	2회차에서 제작 완료, 3회차에서 도구 호출로 변경 예정

1.6 프로젝트 결과

최종적으로 제출해야 할 항목은 아래와 같고 이어지는 프로젝트와 함께 PPTX 형식의 자료와 동작 영상을 촬영 하여 제출한다

■ **산출물과 제출**

1) 웹캠 질문에 YOLO·VLM·손 제스처 인식 결과와 LLM 답변이 대시보드에 표시되는 동작 영상을 제출한다.

2) 최종 제출 시 파일의 명칭은 아래의 형식을 엄격히 준수하여 업로드한다.

◆ AI 코어_2 회차_PJT02_동작시연영상_지역_반_성명 1_성명 2.zip

- 以上